Winter 2026 DSC 240: Homework 0

Zeyu Bian

Update: January 6, 2026

Assignment 1 *Refreshers on Optimization and probability fundamentals.*

(a) (Continuous optimization) Let $x_1, \ldots, x_n \in \mathbb{R}$ and

$$f(\theta) = \sum_{i=1}^{n} w_i (x_i - \theta)^2.$$

Assume first that $w_i > 0$ for all i. Expand and differentiate:

$$f(\theta) = \sum_{i=1}^{n} w_i (x_i^2 - 2x_i\theta + \theta^2) = \Big(\sum_{i=1}^{n} w_i\Big)\theta^2 - 2\Big(\sum_{i=1}^{n} w_i x_i\Big)\theta + \sum_{i=1}^{n} w_i x_i^2.$$

Then

$$f'(\theta) = 2\Big(\sum_{i=1}^{n} w_i\Big)\theta - 2\sum_{i=1}^{n} w_i x_i.$$

Setting $f'(\theta) = 0$ gives

$$\theta^* = \frac{\sum_{i=1}^{n} w_i x_i}{\sum_{i=1}^{n} w_i}.$$

Moreover, when all $w_i > 0$, $\sum_i w_i > 0$ so f is a strictly convex quadratic and θ^* is the unique global minimizer.

If some w_i are negative: $f(\theta)$ is still a quadratic in θ with leading coefficient $a = \sum_{i=1}^{n} w_i$.

- If $\sum_i w_i > 0$, then f still opens upward and the (unique) minimizer is still $\theta^* = \dfrac{\sum_i w_i x_i}{\sum_i w_i}$.
- If $\sum_i w_i = 0$, then $f(\theta)$ becomes linear in θ (unless also $\sum_i w_i x_i = 0$, in which case it is constant). In the generic linear case it has no minimizer over $\mathbb{R}$.
- If $\sum_i w_i < 0$, then f opens downward and is unbounded below, so there is no minimizer.

(b) (Counting and combinatorics) There are $2n$ kids, split uniformly at random into two groups of size n. Consider selecting Group 1 uniformly among all $\binom{2n}{n}$ size-n subsets. Let the two tallest kids be A and B.

- *Same subgroup:* Either both in Group 1 or both in Group 2.

$$\mathbb{P}(\text{both in Group 1}) = \frac{\binom{2n-2}{n-2}}{\binom{2n}{n}}, \qquad \mathbb{P}(\text{both in Group 2}) = \frac{\binom{2n-2}{n}}{\binom{2n}{n}}.$$

Since $\binom{2n-2}{n} = \binom{2n-2}{n-2}$, we get

$$\mathbb{P}(\text{same}) = \frac{2\binom{2n-2}{n-2}}{\binom{2n}{n}} = \frac{n-1}{2n-1}.$$

- *Different subgroups:*

$$\mathbb{P}(\text{different}) = 1 - \mathbb{P}(\text{same}) = 1 - \frac{n-1}{2n-1} = \frac{n}{2n-1}.$$

Check for $n = 2$: $\mathbb{P}(\text{same}) = \frac{1}{3}$ and $\mathbb{P}(\text{different}) = \frac{2}{3}$, which matches a manual enumeration for 4 kids.

(c) (Bayes rule) Let K be the event "candidate knows the answer" and C be the event "candidate answers correctly." Given:

$$\mathbb{P}(K) = p, \quad \mathbb{P}(C \mid K) = 0.99, \quad \mathbb{P}(C \mid K^c) = \frac{1}{k}.$$

First compute $\mathbb{P}(C)$:

$$\mathbb{P}(C) = \mathbb{P}(C \mid K)\mathbb{P}(K) + \mathbb{P}(C \mid K^c)\mathbb{P}(K^c) = 0.99p + \frac{1}{k}(1-p).$$

By Bayes' rule,

$$\mathbb{P}(K \mid C) = \frac{\mathbb{P}(C \mid K)\mathbb{P}(K)}{\mathbb{P}(C)} = \frac{0.99\,p}{0.99\,p + \frac{1}{k}(1-p)}.$$

(d) (Likelihood and maximum likelihood) We have likelihood

$$L(p) = p^6(1-p)^4, \qquad 0 < p < 1.$$

Maximizing $\log L(p)$ preserves the $\arg\max$ because $\log(\cdot)$ is strictly increasing:

$$\arg\max_p L(p) \;=\; \arg\max_p \log L(p).$$

Now

$$\ell(p) := \log L(p) = 6\log p + 4\log(1-p).$$

Differentiate and set to zero:

$$\ell'(p) = \frac{6}{p} - \frac{4}{1-p} = 0 \implies 6(1-p) = 4p \implies 6 = 10p \implies p^* = \frac{6}{10} = 0.6.$$

Also $\ell''(p) = -\frac{6}{p^2} - \frac{4}{(1-p)^2} < 0$, so this is indeed the (unique) maximizer.

(e) (Calculus, gradients) Let $w \in \mathbb{R}^d$, $x_i \in \mathbb{R}^d$ (column vectors), $y_i \in \mathbb{R}$, and $\lambda \geq 0$.

$$F(w) = \sum_{i=1}^{n} (x_i^T w - y_i)^2 + \lambda \sum_{j=1}^{d} w_j^2.$$

For each i, define $r_i(w) = x_i^T w - y_i$ (a scalar). Then $\nabla_w r_i(w) = x_i$. By the chain rule,

$$\nabla_w \big(r_i(w)^2\big) = 2 r_i(w) \nabla_w r_i(w) = 2(x_i^T w - y_i) x_i.$$

Also $\nabla_w \big(\lambda \sum_{j=1}^{d} w_j^2\big) = 2\lambda w$. Therefore,

$$\nabla F(w) = 2 \sum_{i=1}^{n} (x_i^T w - y_i) x_i + 2\lambda w.$$

Matrix form (optional): If $X \in \mathbb{R}^{n \times d}$ has rows x_i^T and $y \in \mathbb{R}^n$ has entries y_i, then

$$F(w) = \|Xw - y\|_2^2 + \lambda \|w\|_2^2, \qquad \nabla F(w) = 2X^T(Xw - y) + 2\lambda w.$$

(f) (Chain-rule and softmax / log-sum-exp) Let

$$f(x_1, \ldots, x_n) = \log \left(\sum_{i=1}^{n} e^{x_i} \right).$$

Let $S = \sum_{i=1}^{n} e^{x_i}$. Then

$$\frac{\partial f}{\partial x_j} = \frac{1}{S} \cdot \frac{\partial}{\partial x_j} \left(\sum_{i=1}^{n} e^{x_i} \right) = \frac{1}{S} e^{x_j}.$$

Thus the gradient is

$$\nabla f(x) = \begin{bmatrix} \frac{e^{x_1}}{\sum_{i=1}^{n} e^{x_i}} \\ \vdots \\ \frac{e^{x_n}}{\sum_{i=1}^{n} e^{x_i}} \end{bmatrix},$$

which is exactly the *softmax* probability vector.

(g) (Simple mathematical proof) Let $m = \max_i x_i$.

Lower bound: Since $e^{x_i} \geq 0$ and one term attains e^m,

$$\sum_{i=1}^{n} e^{x_i} \geq e^m \quad \Longrightarrow \quad f(x) = \log \left(\sum_{i=1}^{n} e^{x_i} \right) \geq \log(e^m) = m.$$

Upper bound: Since $x_i \leq m$ for all i, we have $e^{x_i} \leq e^m$ for all i. Therefore,

$$\sum_{i=1}^{n} e^{x_i} \leq \sum_{i=1}^{n} e^m = n e^m \quad \Longrightarrow \quad f(x) = \log \left(\sum_{i=1}^{n} e^{x_i} \right) \leq \log(n e^m) = m + \log n.$$

Combining both inequalities yields

$$\max_i x_i \;\leq\; f(x_1, \ldots, x_n) \;\leq\; \max_i x_i + \log n.$$

Assignment 2 *Refreshers on time complexity, data structure and python / numpy. Please write codes for python3.*

(a) (Numpy matrix creation and multiplication) Construct:

$$A \in \mathbb{R}^{5\times 4} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 \end{bmatrix}, \qquad B \in \mathbb{R}^{4\times 3} = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix},$$

where B is filled *top-to-bottom, then left-to-right* (column-major order).

```python
import numpy as np

def make_A_B():
    # A: 1..20 left-to-right then top-to-bottom (row-major fill)
    A = np.arange(1, 21).reshape(5, 4)

    # B: 1..12 top-to-bottom then left-to-right (column-major fill)
    B = np.arange(1, 13).reshape(4, 3, order="F")
    return A, B

def matmul(A, B):
    """Return the matrix product AB."""
    return A @ B # equivalently: np.dot(A, B)

# Example:
# A, B = make_A_B()
# C = matmul(A, B)
```

(b) (Sparse matrix-vector multiplication) Dense case: Multiplying an $n \times n$ dense matrix by a dense vector costs

$$O(n^2)$$

time in the worst case (because there are n^2 multiply-add contributions).

Sparse case: If A is sparse with $\mathrm{nnz}(A)$ nonzero entries, sparse matrix-vector multiplication costs

$$O(\mathrm{nnz}(A))$$

time (up to constant factors depending on storage format), since you only process nonzero entries.

Construct $A \in \mathbb{R}^{(n-1)\times n}$ so that $Ax = [x_1 - x_2, \ldots, x_{n-1} - x_n]^T$: This is the first-difference operator:

$$A = \begin{bmatrix} 1 & -1 & 0 & \cdots & 0 \\ 0 & 1 & -1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & -1 \end{bmatrix}.$$

```python
import numpy as np
from scipy import sparse

def difference_matrix(n: int):
    """
    Return a sparse matrix A of shape ((n-1), n) such that
    for any x in R^n, A @ x = [x1-x2, x2-x3, ..., x_{n-1}-x_n]^T.
    """
    if n < 2:
        raise ValueError("n must be >= 2 so that (n-1) x n is non-empty.")

    data = np.ones(n - 1)
    # diagonal 0 has +1's, diagonal +1 has -1's
    A = sparse.diags([data, -data], offsets=[0, 1], shape=(n - 1, n),
        format="csr")
    return A

# Example check:
# n = 5
# A = difference_matrix(n)
# x = np.array([1, 3, 6, 10, 15])
# print(A @ x) # should be [1-3, 3-6, 6-10, 10-15] = [-2, -3, -4, -5]
```

(c) (Manipulating text using python, data structures) We can use a **dictionary** (`dict`) mapping each word to its count. For basic tokenization, one common approach is to lowercase and extract alphanumeric "words" using a regex.

```python
import re

def word_counts(filepath: str):
    """
    Read a text file line by line and return a dictionary:
    {word -> count}.
    Words are case-folded to lowercase and extracted via regex.
```

```
    """
    counts = {} # dict: word -> int count

    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            # Extract tokens (letters/digits/underscore). You may adjust
                pattern if desired.
            words = re.findall(r"\w+", line.lower())
            for w in words:
                counts[w] = counts.get(w, 0) + 1

    return counts

# Example:
# counts = word_counts("data_example.txt")
# print(len(counts), "unique words")
# print(counts.get("machine", 0))
```

Data structures used: A `dict` for counts (hash table), plus temporary `list` of tokens per line from `re.findall`.

Time complexity: Let T be the total number of word tokens extracted from the file. Each dictionary increment is $O(1)$ expected time, so total time is $O(T)$ (plus the linear pass to read and tokenize the text, which is also linear in the input size).

Space complexity: Let U be the number of unique words. The dictionary stores U keys, so space is $O(U)$ (ignoring the input file storage, and treating tokenization overhead per line as transient).

(d) (Recursion with bookkeeping in python) Define Fibonacci numbers with $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. A memoized recursion computes each F_m at most once.

```
# Global memo dictionary for bookkeeping:
_fib_memo = {0: 0, 1: 1}

def fibonacci(n: int) -> int:
    """
    Return the nth Fibonacci number using recursion + memoization.
    Uses convention: F_0=0, F_1=1.
    """
    if n < 0:
        raise ValueError("n must be nonnegative.")
```

```
    if n in _fib_memo:
        return _fib_memo[n]
    _fib_memo[n] = fibonacci(n - 1) + fibonacci(n - 2)
    return _fib_memo[n]

# Example:
# print(fibonacci(10)) # 55
```

Time complexity (with bookkeeping): $O(n)$, since each value $F_0, \ldots, F_n$ is computed once and stored.

Space complexity: $O(n)$ for the memo table (and also $O(n)$ worst-case recursion depth).